

AWS Serverless vs Server

The Complete Comparison Guide

Features • Pricing • Best Use Cases • Winner Per Scenario

■ What Is What?

■■ SERVERLESS (AWS manages everything)

- No servers to provision, patch, or manage
- Scales automatically from 0 to millions
- Pay only when code runs — no idle cost
- AWS handles OS, runtime, infrastructure
- Deploy code or config — nothing else

■■ SERVER-BASED (You manage compute)

- You provision and maintain the machine
- Full control over OS, runtime, packages
- Always running — pay 24/7 even if idle
- Scale requires planning or Auto Scaling
- Best for long-running or custom workloads

■ AWS Services: Serverless vs Server

Category	Serverless Service	Server-Based Service	Best Pick
Compute	AWS Lambda	EC2 (Virtual Machine)	Lambda for events EC2 for 24/7 apps
Containers	AWS Fargate	ECS on EC2 / EKS on EC2	Fargate for simplicity EC2 for cost at scale
Database (SQL)	Aurora Serverless v2	RDS (MySQL/PostgreSQL)	Aurora Serverless for variable traffic
Database (NoSQL)	DynamoDB (On-Demand)	DynamoDB (Provisioned)	On-Demand for unpredictable Provisioned for steady load
Storage	S3 (always serverless)	EBS (attached to EC2)	S3 for objects EBS for block storage
API Layer	API Gateway + Lambda	ALB + EC2 / ECS	API GW+Lambda for low traffic; ALB for high
Messaging	SQS / SNS / EventBridge	Kafka on EC2 (MSK)	SQS/SNS for simplicity MSK for heavy streaming
Search	OpenSearch Serverless	OpenSearch Service (EC2)	Serverless for dev/test EC2 for production search
Analytics	Athena + Glue Serverless	EMR on EC2	Athena for SQL on S3 EMR for Spark/Hadoop

Category	Serverless Service	Server-Based Service	Best Pick
CI/CD Compute	AWS CodeBuild	Jenkins on EC2	CodeBuild for AWS-native Jenkins for flexibility

■ Feature Comparison

Feature	Serverless	Server-Based	Winner
Setup Time	■ Minutes — just deploy code	■ Hours — provision, configure OS	Serverless
Scaling	■ Auto, instant, 0 to millions	■ Manual or Auto Scaling (slower)	Serverless
Availability	■ AWS managed, multi-AZ by default	■ You configure HA and Multi-AZ	Serverless
Execution Time	■ Max 15 min (Lambda)	∞ Unlimited — runs as long as needed	Server-Based
Cold Start Latency	*■ ~100ms–1s first invocation	■ No cold start — always warm	Server-Based
OS/Runtime Control	■ AWS chooses runtime/OS	■ Full control — any OS, any version	Server-Based
Custom Packages	■ Via Lambda Layers (limited)	■ Install anything on EC2	Server-Based
Persistent Connections	■ Stateless — no keep-alive DB conn	■ Connection pools, persistent sockets	Server-Based
Observability	■ Auto CloudWatch Logs + metrics	■ CloudWatch Agent needed (more setup)	Tie
Security Patching	■ AWS patches runtime automatically	■ You patch OS and runtime yourself	Serverless
Vendor Lock-in	■ AWS-specific APIs (Lambda, etc.)	■ Portable containers or standard VMs	Server-Based
Team Skills Needed	■ App dev only — no ops needed	■ Requires DevOps / infra knowledge	Serverless

■ Pricing Comparison (AWS us-east-1)

Service	Serverless Cost	Server Cost	Cost Winner
Compute (Light Traffic)	Lambda: FREE 1M req/mo \$0.20 per 1M req after \$0.0000166 per GB-sec	EC2 t3.micro: ~\$8/mo (always running, 24/7)	■ Serverless (saves ~\$7/mo)
Compute (Heavy Traffic)	Lambda: \$0.20 per 1M req 10M req = ~\$2 + duration Can exceed EC2 cost	EC2 t3.large: ~\$60/mo Handles millions of req Predictable flat cost	■ Server-Based (predictable at scale)

Service	Serverless Cost	Server Cost	Cost Winner
Database (Variable Traffic)	Aurora Serverless v2: ~\$0.12/hr per ACU Scales to 0 when idle	RDS db.t3.medium: ~\$50–70/mo fixed Running 24/7	■ Serverless (idle = ~\$0)
Database (Constant High Load)	DynamoDB On-Demand: \$1.25 per million writes \$0.25 per million reads	DynamoDB Provisioned: ~\$0.00065/WCU/hr 30–60% cheaper at steady load	■ Server-Based (Provisioned cheaper)
Storage	S3 Standard: \$0.023/GB/month No minimum, no compute	EBS gp3: \$0.08/GB/month Needs EC2 attached	■ Serverless (S3) 4x cheaper per GB
Container Orchestration	Fargate: \$0.04048/vCPU/hr \$0.004445/GB/hr (no node management)	EKS on EC2: \$0.10/hr cluster fee + EC2 cost (~\$30-60/mo) Cheaper at scale	Depends on scale Fargate < 10 tasks EC2 > 10 tasks
API Layer	API Gateway: \$3.50 per million calls + Lambda cost	ALB: ~\$22/mo base + \$0.008 per LCU Cheaper at high volume	ALB+EC2 cheaper over 5M req/month

* All prices approximate as of 2025, us-east-1 region. Always verify on aws.amazon.com/pricing

■ When to Use What — Best Pick by Scenario

Scenario	Best Choice	Why
■ Startup / MVP	Serverless (Lambda + DynamoDB + API GW)	Zero infrastructure setup. Pay only for real usage. Ship fast.
■ High-Traffic Web App (>10M req/day)	Server-Based (EC2/ECS + RDS + ALB)	Predictable flat cost. No cold starts. Connection pooling.
■ Scheduled Jobs / Cron Tasks	Serverless (Lambda + EventBridge)	Runs only when needed. Zero idle cost. Auto retry on failure.
■ Event-Driven Processing	Serverless (Lambda + SQS/SNS/S3 trigger)	Pay per event. Scales to 0. Perfect for S3 file processing.
■ Enterprise / Compliance Workloads	Server-Based (EC2 + RDS Multi-AZ + VPC)	Full OS control. Custom security agents. Compliance auditing.
■ ML Model Serving (High Frequency)	Server-Based (EC2 GPU or ECS on EC2)	No 15-min timeout. Warm model in memory. Consistent latency.
■ Data Analytics Pipeline	Serverless (Athena + Glue + S3)	Pay per query. No cluster management. Scale is automatic.
■ Real-Time WebSocket App	Server-Based (EC2/ECS + ALB WebSocket)	Lambda has 29s connection limit. EC2 supports persistent sockets.
■ Dev / Test Environments	Serverless (Lambda) or Fargate	Scales to 0 overnight. No cost when not in use. Fast teardown.

Scenario	Best Choice	Why
■ Global Multi-Region App	Serverless (Lambda@Edge + CloudFront + DynamoDB Global Tables)	Deploy code to 400+ edge locations. Sub-millisecond global reads.

■ THE GOLDEN RULE

Use **Serverless** when traffic is unpredictable, low, or event-driven — and you want zero ops overhead.

Use **Server-Based** when traffic is steady, high-volume, needs long-running processes, custom runtimes, or persistent connections.

■ **Most production systems use BOTH — Serverless for event processing, Server-Based for core application workloads.**